Choosing the Stack

Immersion Week SF · Day 2 · Session 2.1

35

Minutes

You leave with a tool per step

What This Session Covers

01 Give every step a named tool

02 Decide what to stub, not build

03 RAG or just more context

04 Stand up the smallest layer

What will you not build?

From 1.3 to Now

A board

Who does each step

vs

A stack

What each step runs on

How We Get There

01

Map

Reopen yesterday's diagram

02

Tool

Name one per step

03

Stub

Mark what stays fake

04

Retrieve

Only if you must

"Boring tools, interesting problem."

- The Day 2 rule

01

A Tool for Every Step

Low-code

Assembling built pieces instead of writing them

The Rule for This Hour

Every box gets one tool name

No box may stay undecided

"We fake it" counts as decided

Write it on the board now

Where the Tool Names Come From

What your team already uses

What the workshop hands you

What has a free tier today

Not what you meant to learn

The Four Jobs

Job	What it covers
Intake	Getting the input in
Think	The model doing the work
Store	Anything you look up later
Show	What the user actually sees

A Worked Mapping

Step	Tool
Collect the input	A form or a paste box
Understand it	Claude, one prompt
Look something up	Claude Project files
Produce the output	Claude Artifact
Show a stranger	A hosted page

Signs You Picked Wrong

Nobody on the team has used it

Setup is measured in hours

It needs a key you don't have

The docs assume production

Build or Stub

Build it

It is the thing you claim

The demo touches it live

Nobody believes it faked

Stub it

It is plumbing, not proof

The demo never opens it

It works but is boring

Stub

A fake that is honest about being fake

How to Decide, Per Box

01 Ask what it proves

If nothing, it is plumbing

02 Ask who sees it

If nobody, stub it

03 Ask what it costs

Over an hour needs a reason

Always Safe to Stub

Login and user accounts

Payments of any kind

Email and notifications

Anything with a real database

Never Safe to Stub

The step your idea rests on

Whatever you claim is novel

The output a stranger reads

The path the demo walks

Three Tests Before You Commit

Under an hour

You can have it running before lunch

Someone knows it

One person on the team has shipped with it

Fails quietly

When it breaks, the demo still walks

Picking Between Two Options

	Familiar tool	Better tool
Setup	**Minutes**	Half a day
Ceiling	Low	High
Demo risk	**Low**	Unknown

Where the Time Actually Goes

80%

Wiring things together

Not the clever part

20%

The idea itself

What you are judged on

02

RAG or Just Context

RAG

Fetch the few relevant bits, then ask

Does it all fit already?

Two Ways to Give Context

Paste it

Everything, every time

vs

Fetch it

Only what is relevant

What a Context Window Holds

A long policy document

A few hundred support tickets

A codebase of modest size

More than you probably have

The Decision, in Order

01 Measure the corpus

Actual size, not your guess

02 Try pasting it

If it fits, you are done

03 Only then, retrieve

And keep it as small as you can

Which One Do You Need

Just context	Real retrieval
It fits in one window	Far more than fits
The corpus barely changes	It changes constantly
Every question sees it all	Questions need a slice

What Each One Costs You

	Paste it	Retrieve it
Build time	Zero	Hours
Per question	More tokens	Fewer tokens
Breaks when	Corpus grows	Search misses
Debuggable	Easily	Less easily

The Costs Nobody Budgets

Chunking the documents well

Deciding how many to fetch

Debugging what it missed

Explaining it in the demo

Measure before you architect

```
$ wc -w docs/*.md
  41,204 total

# ~55k tokens. It fits.
# No retrieval needed today.
```

You Are Over-Building If

You picked a vector database first

Your corpus is under 20 files

You cannot name a missed query

It is not tomorrow's demo step

"The cheapest thing that works."

- Objective two, in five words

03

The Smallest Layer

Claude Project

A workspace with your files always in context

Why It Is the Baseline

Standing it up takes minutes

No keys, no store, no code

Your files are simply there

Anyone can drive it

Stand One Up

01 Create the Project

Name it after the problem

02 Add your documents

The real ones, not samples

03 Write the instructions

Say how to answer, and how not to

Project instructions worth writing

Answer only from the attached files.
Quote the filename you used.
If the files do not cover it,
say so plainly. Do not guess.

What Goes in the Project

The documents you must cite

One example of a good answer

Your glossary and odd terms

Nothing you would not ship

The Baseline to Beat

Measure	Your target
Questions answered	More than the Project
Wrong answers	Fewer than the Project
Time to build	Under one afternoon
Teammate can run it	Yes

How to Know You Beat It

01 Write ten questions

Real ones, before you build

02 Run both, same questions

Project first, then yours

03 Count, do not vibe

Keep whichever wins

Three Layers, Cheapest First

Paste it in

Zero build; use it while the corpus is small

Claude Project

Minutes to stand up; the baseline to beat

Real retrieval

Only once you can name what the others missed

When to Graduate

The Project cannot hold it all

You can name what it missed

The corpus changes daily

You have the afternoon spare

Two Teams, Same Corpus

Built retrieval	Used a Project
Afternoon on chunking	Twenty minutes total
Demo has a search step	Demo has an answer
Nobody else can run it	Whole team can run it

Leaving This Session

Every box has a tool name

Every box is build or stub

Retrieval decided, not deferred

The baseline is standing up

Write This Down Now

Question	Your answer
Riskiest tool choice	…
Biggest stub	…
Retrieval: yes or no	…
What proves it works	…

Can you start building?

Next: Building the Happy Path

One known-good input, end to end

No edge cases, no error handling

Claude Code drives each step

Commit the moment it works